CS V 21-08-2025 Register Number: ……..

Toc H INSTITUTE OF SCIENCE & TECHNOLOGY TIST

B. Tech Degree Internal Assessment Test – I, August 2025

CST 309 Management of Software Systems

ANSWER SCHEME

Time: 1.5 Hours **Maximum Marks: 50**

PART A

(Answer all questions)

1. **Outline the advantages of an incremental development model over a waterfall model.**

Any 3 advantages with explanation 1 mark each.

1) The cost of accommodating changing customer requirements is reduced.
   The amount of analysis and documentation that has to be redone is much less than is required with the waterfall model.
2) It is easier to get customer feedback on the development work that has been done.
   Customers can comment on demonstrations of the software and see how much has been implemented.
3) More rapid delivery and deployment of useful software to the customer is possible.
   Customers are able to use and gain value from the software earlier than is possible with a waterfall process.

2. **Explain the essential attributes of good software**. (3)

Any 3 attributes with explanation 1 mark each.

Maintainability	Software should be written in such a way so that it can evolve to meet the changing needs of customers. This is a critical attribute because software change is an inevitable requirement of a changing business environment.

Dependability and security	Software dependability includes a range of characteristics including reliability, security and safety. Dependable software should not cause physical or economic damage in the event of system failure. Malicious users should not be able to access or damage the system.
Efficiency	Software should not make wasteful use of system resources such as memory and processor cycles. Efficiency therefore includes responsiveness, processing time, memory utilisation, etc.
Acceptability	Software must be acceptable to the type of users for which it is designed. This means that it must be understandable, usable and compatible with other systems that they use.

3. **Identify the benefits of using prototyping in software development**. (3)

Any 3 benefits with explanation – 1 mark each.

- Prototyping in software development refers to creating a working model or early version of a system to demonstrate and test its features before the final version is developed. There are several significant benefits to using prototyping in the software development process:

1. Early User Feedback

- **Benefit**: Prototyping allows stakeholders, especially users, to interact with a working model of the software early in the development process.

2. Improved Requirement Clarification

- **Benefit**: Through interactive prototypes, the development team can refine and clarify requirements based on real user input and feedback.

3. Faster Time to Market

- **Benefit**: Prototyping allows developers to quickly build a basic version of the software that can be tested and iterated upon.

4. Reduced Risk of Failure

- **Benefit**: Since prototyping allows for iterative testing and validation, there is a reduced risk of major failures in the final product.

5. Enhanced Communication

- **Benefit**: Prototypes serve as a visual tool to communicate the functionality and design of the system to stakeholders, including clients, designers, developers, and end-users.

4. **Differentiate between functional and non-functional requirement**s. (3)

- **Functional requirements – 1.5 marks**
  - Statements of services the system should provide, how the system should react to particular inputs and how the system should behave in particular situations.
  - May state what the system should not do.
- **Non-functional requirements – 1.5 marks**
  - Constraints on the services or functions offered by the system such as timing constraints, constraints on the development process, standards, etc.
  - Often apply to the system as a whole rather than individual features or services

5. **Develop a Use Case Diagram for Student Attendance System.** (3)

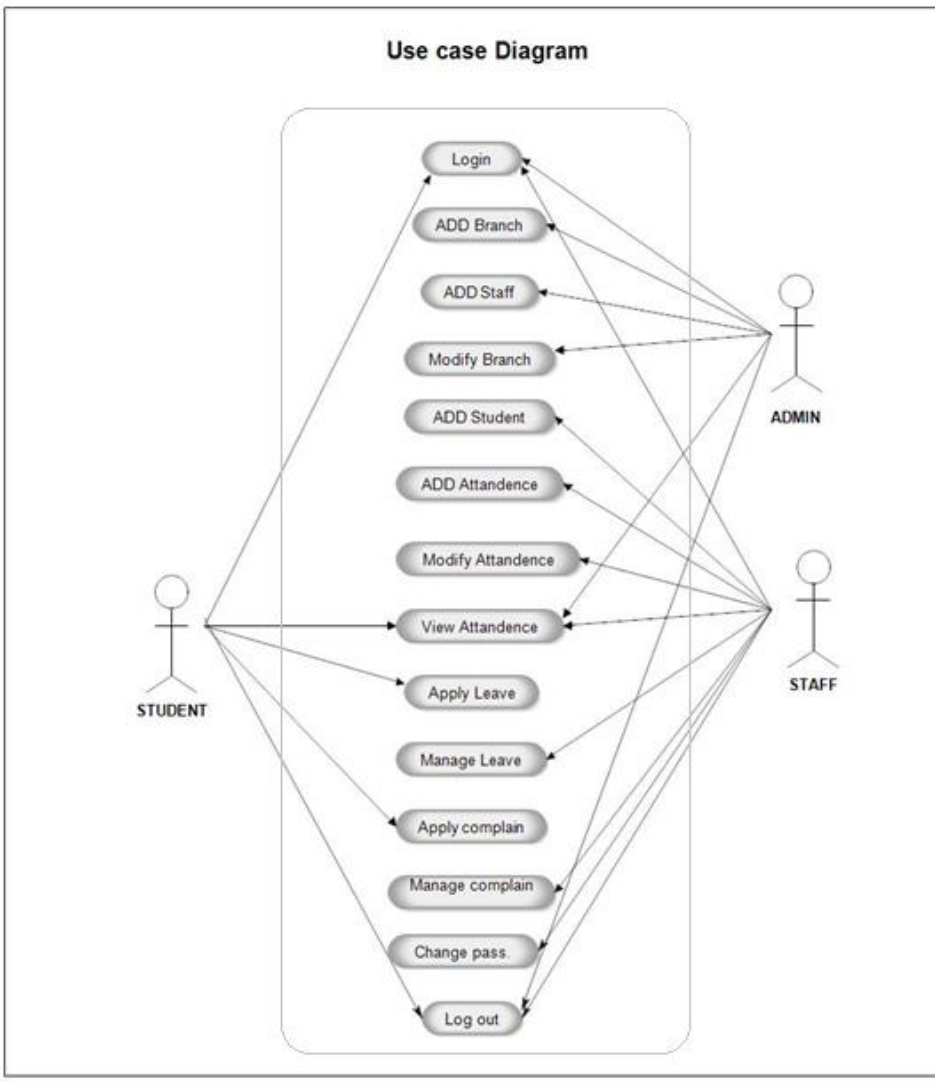
Diagram with any 3 functions – 1 mark each

Key Actors:

1. **Student** – The individual whose attendance is being tracked.
2. **Teacher/Instructor** – The person who manages attendance and records it.
3. **Administrator** – The user responsible for managing the system, including adding/removing users, and generating reports.
4. **System** – The application itself that manages the attendance records and performs backend processes.

Key Use Cases:

1. **Mark Attendance** – Teacher marks the attendance for the students.
2. **View Attendance** – Students and teachers can view attendance records.
3. **Generate Attendance Report** – The administrator can generate reports based on attendance data.
4. **Login/Logout** – Students, Teachers, and Administrators log into the system.
5. **Manage Users** – The administrator adds or removes users (students, teachers).
6. **Update Attendance** – Teacher can update or modify attendance records.
7. **Receive Notifications** – Students and teachers can receive notifications (e.g., for absences).
8. **View Personal Attendance** – Students can view their own attendance record.



PART B

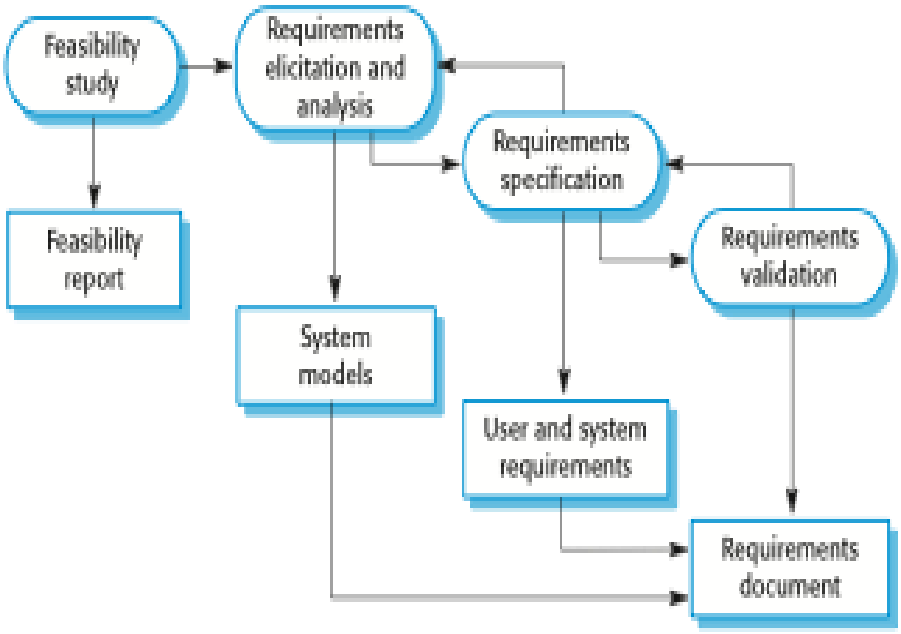
(Answer all questions)

6.(i) **Illustrate software process activities with an example.** (7)

Software specification – 2 marks

- **Software specification**, where customers and engineers define the software that is to be produced and the constraints on its operation.
- The process of establishing and defining what services are required from the system and identifying the constraints on the system's operation and development.
- Is a particularly critical stage of the software process as errors at this stage inevitably lead to later problems in system design and implementation.

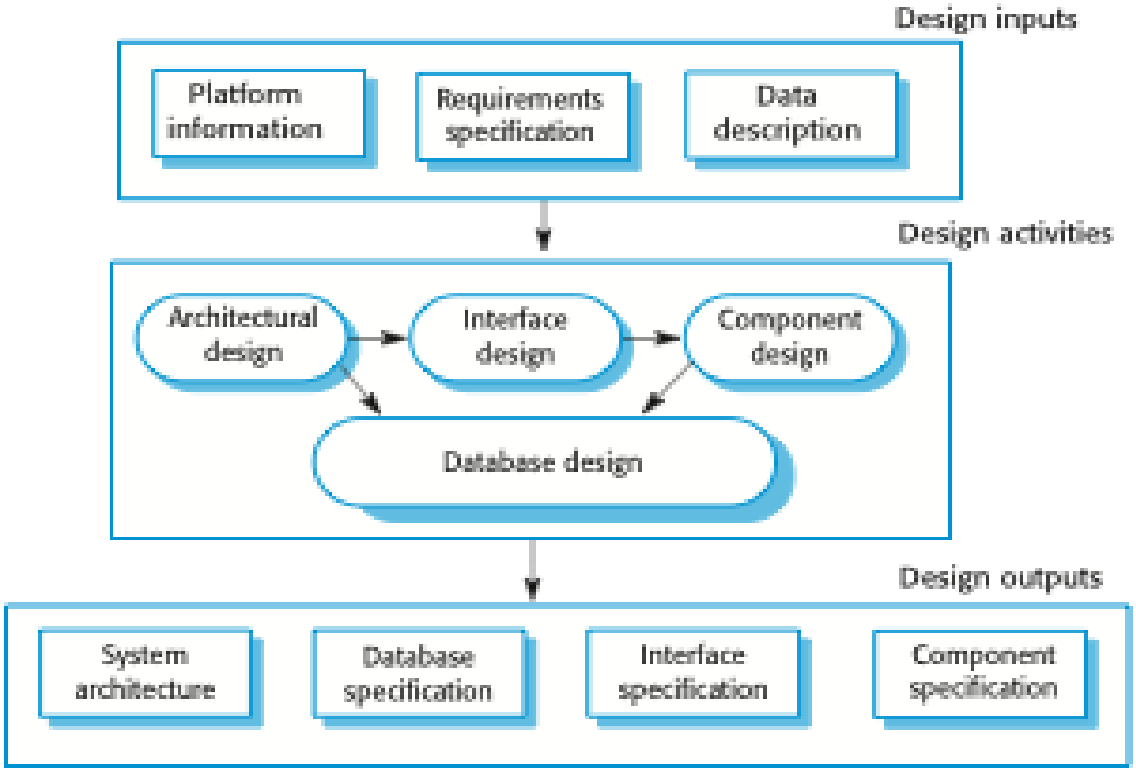
- RE process aims to produce an agreed requirements document that specifies a system satisfying stakeholder requirements.



Software development – 2 marks

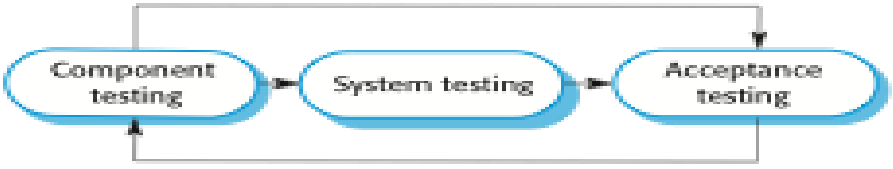
where the software is designed and programmed.

- Software design
  - Design a software structure that realises the specification;
- Implementation
  - Translate this structure into an executable program;
- The activities of design and implementation are closely related and may be inter-leaved.



- **Software validation – 2 marks**
  - Verification and validation (V & V) is intended to show that a system conforms to its specification and meets the requirements of the system customer.
  - Involves checking processes such as inspections and reviews.
  - System testing involves executing the system with test cases that are derived from the specification of the real data to be processed by the system.
  - Testing is the most commonly used V & V activity.

Stages of Testing



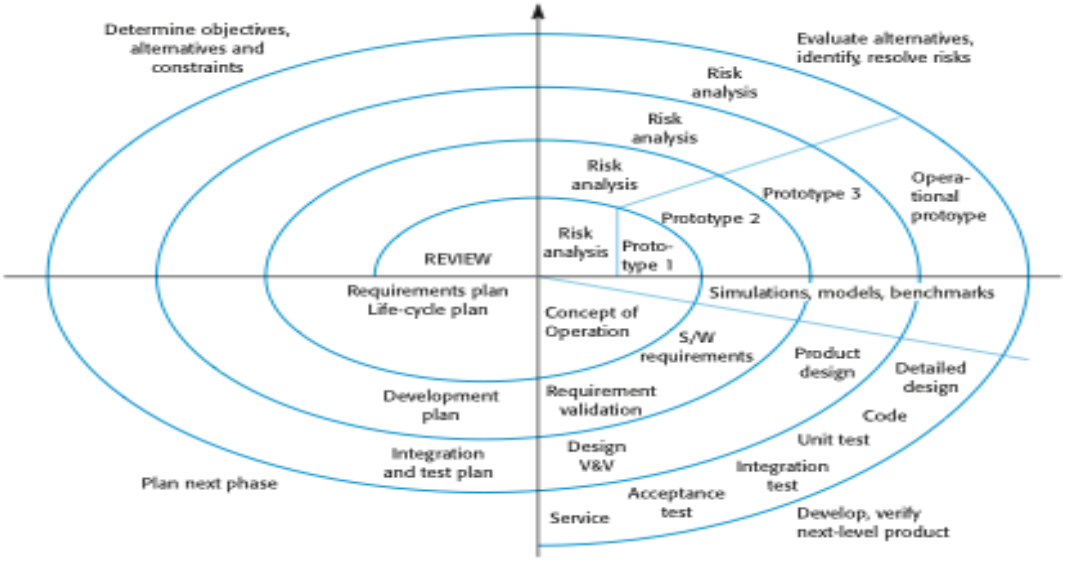
Software evolution – 1 marks

- where the software is modified to reflect changing customer and market requirements.
- Software is inherently flexible and can change.
- As requirements change through changing business circumstances, the software that supports the business must also evolve and change.
- Although there has been a demarcation between development and evolution (maintenance) this is increasingly irrelevant as fewer and fewer systems are completely new.

(ii) **You are given a project which involves many risks, that are difficult to anticipate at the start of the project. Design suitable lifecycle model for the project along with its importance.** (7)

Identified the model - <u>Boehm's spiral model</u> – 1 mark

Diagram – 3 marks



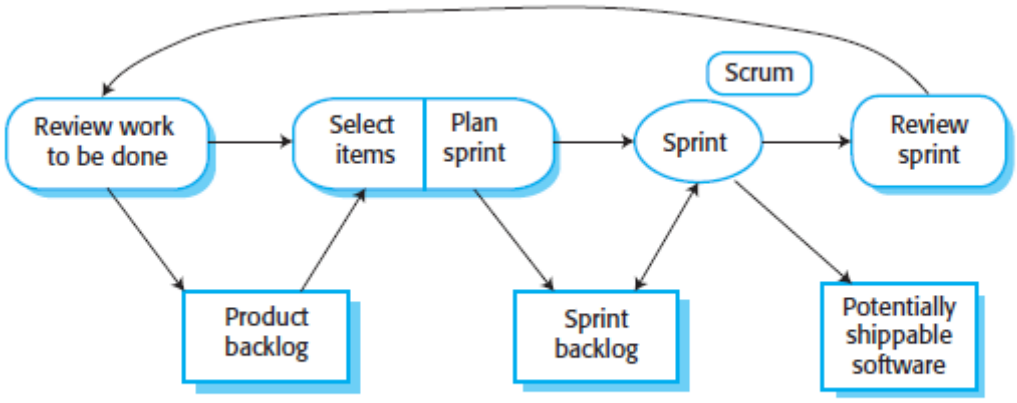
Each loop in the Spiral Model is split into 4 Sectors – 3 marks

- **Objective setting**
  - Specific objectives for the phase are identified. Constraints on the process and the product are identified and a detailed management plan is drawn up. Project risks are identified. Alternative strategies may planned.
- **Risk assessment and reduction**
  - Risks are assessed and activities put in place to reduce the key risks.
- **Development and validation**
  - A development model for the system is chosen which can be any of the generic models.
- **Planning** (7)

- The project is reviewed and the next phase of the spiral is planned.

7.(i) Explain the Scrum process with a neat diagram.

Diagram – 3 marks



Explanation – 4 marks

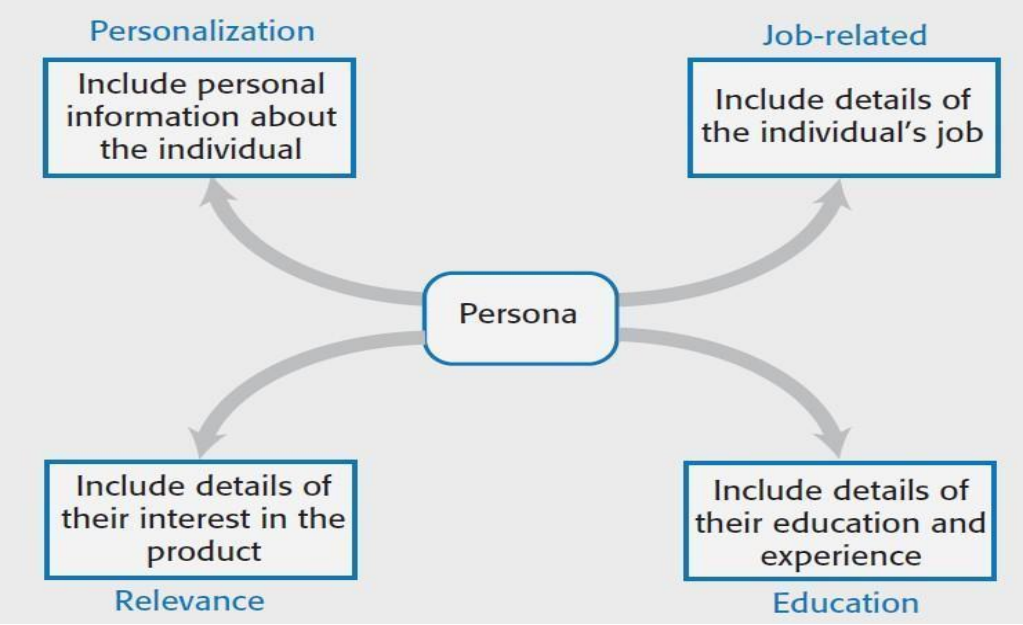
- The 'Scrum master' is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.
- The whole team attends short daily meetings (scrum)where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the following day.
- This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them, there is no top-down direction from the Scrum Master.
- Everyone participates in this short-term planning; the daily interactions among Scrum teams may be coordinated using a Scrum board. This is an office whiteboard that includes information and post-it notes about the Sprint backlog, work done, unavailability of staff, and so on. This is a shared resource for the whole team, and anyone can change or move items on the board. It means that any team member can, at a glance, see what others are doing and what work remains to be done.
- At the end of each sprint, there is a review meeting, which involves the whole team. This meeting has two purposes. First, it is a means of process

improvement. The team reviews the way they have worked and reflects on how things could have been done better. Second, it provides input on the product and the product state for the product backlog review that precedes the next sprint.

8. (i) Describe Personas, Scenarios and Feature identification. (7)

Personas – 2.5 marks

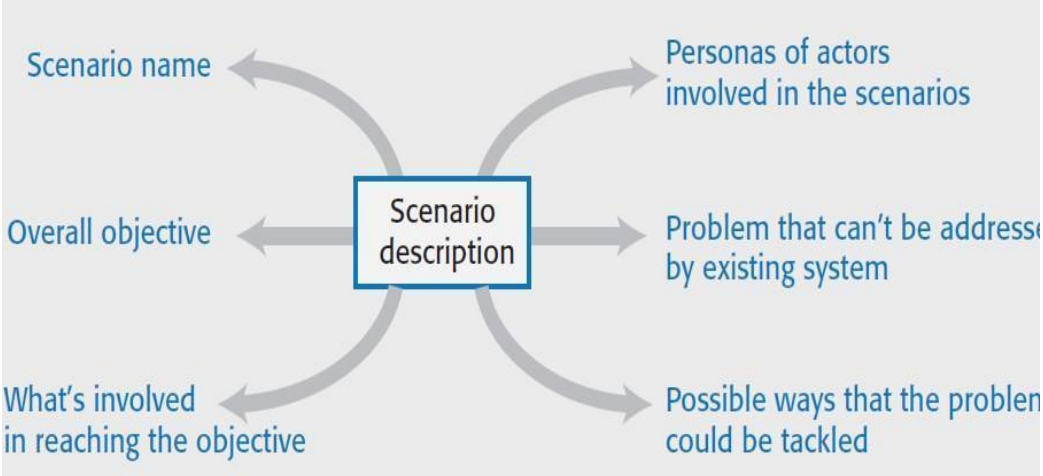
- Personas are 'imagined users' where you create a character portrait of a type of user that you think might use your product.
- For example, if your product is aimed at managing appointments for dentists, you might create a dentist persona, a receptionist persona and a patient persona.
- Personas of different types of user help you imagine what these users may want to do with your software and how it might be used. They help you envisage difficulties that they might have in understanding and using product features.



Scenarios – 2.5 marks

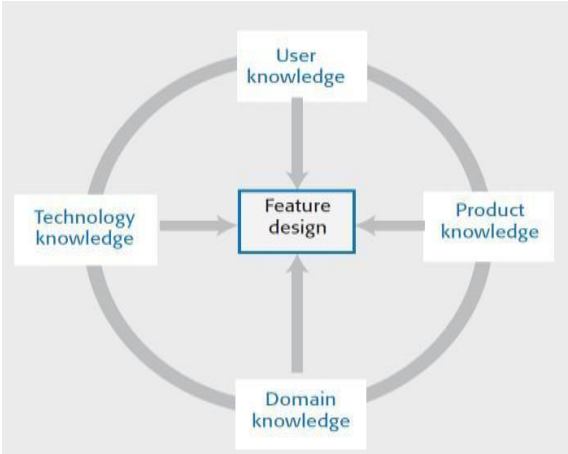
- A scenario is a narrative that describes how a user, or a group of users, might use your system.
- There is no need to include everything in a scenario – the scenario isn't a system specification.

- It is simply a description of a situation where a user is using your product's features to do something that they want to do.
- Scenario descriptions may vary in length from two to three paragraphs up to a page of text.



Feature identification – 2 marks

- Your aim in the initial stage of product design should be to create a list of features that define your product.
- A feature is a way of allowing users to access and use your product's functionality so the feature list defines the overall functionality of the system.

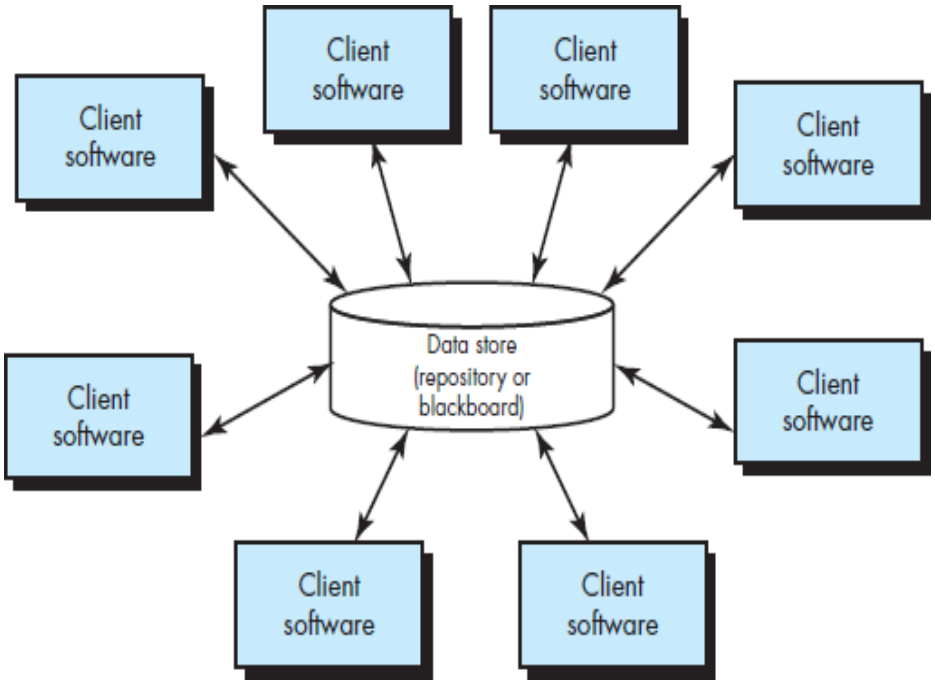


(ii) Illustrate any two software architectural styles. (7)

Any 2 software architectural with diagram and explanation 3.5 marks each

1. **Data Centred Architecture:**

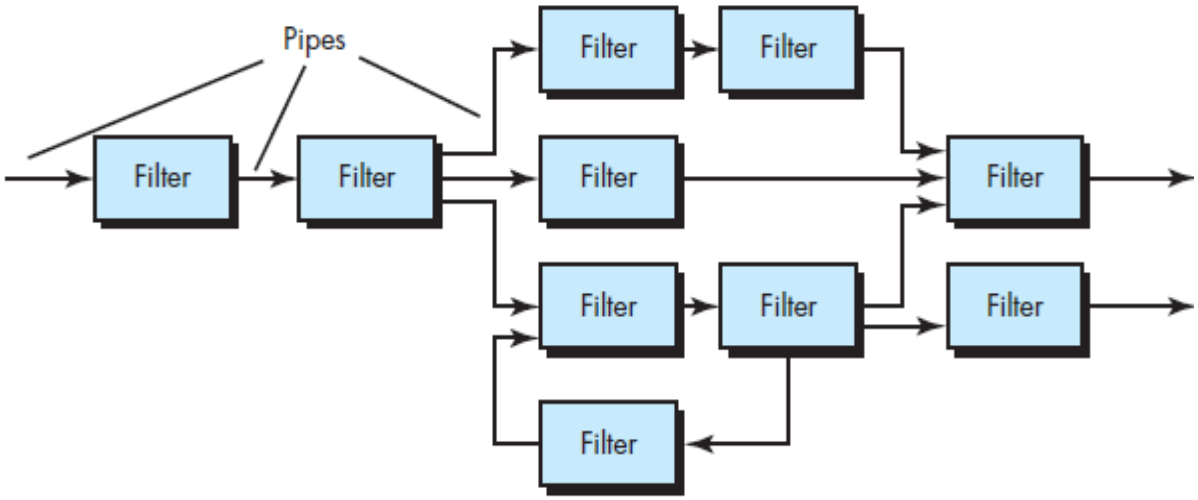
- A data store (e.g., a fi le or database) resides at the center of this architecture and is accessed frequently by other components that update, add, delete, or otherwise modify data within the store. Figure 13.1 illustrates a typical data-centered style.
- Data-centered architectures promote integrability.
- That is, existing components can be changed and new client components added to the architecture without concern about other clients (because the client components operate independently).
- In addition, data can be passed among clients using the blackboard mechanism (i.e., the blackboard component serves to coordinate the transfer of information between clients).
- Client components independently execute processes.



2. **Data Flow Architecture:**

- This architecture is applied when input data are to be transformed through a series of computational or manipulative components into output data. A pipe-and-filter pattern has a set of components, called filters , connected by pipes that transmit data from one component to the next.
- Each filter works independently of those components upstream and downstream, is designed to expect data input of a certain form, and produces data output (to the next filter) of a specified form.

- However, the filter does not require knowledge of the workings of its neighboring filters. If the data flow degenerates into a single line of transforms, it is termed batch sequential .
- This structure accepts a batch of data and then applies a series of sequential components (filters) to transform it.

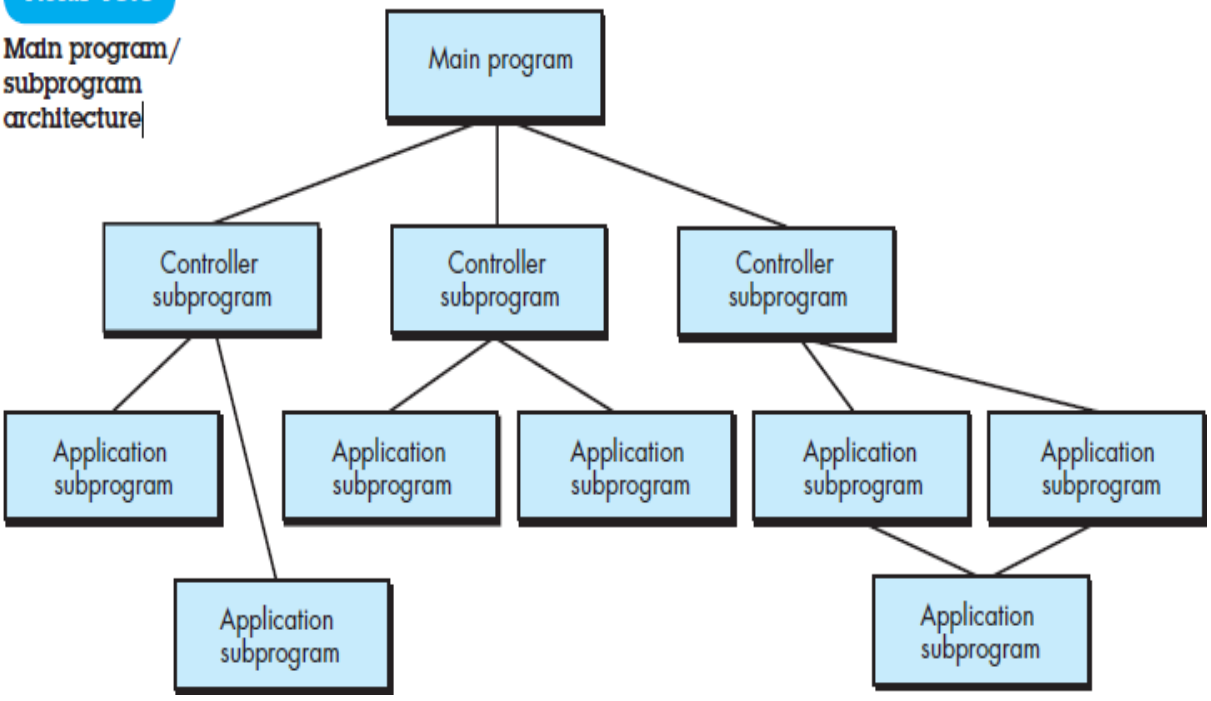


Pipes and filters

3. Call and Return Architecture

- This architectural style enables you to achieve a program structure that is relatively easy to modify and scale. A number of substyles exist within this category:
- Main program/subprogram architectures. This classic program structure decomposes function into a control hierarchy where a "main" program invokes a number of program components, which in turn may invoke still other components. Figure 13.3 illustrates an architecture of this type.
- Remote procedure call architectures. The components of a main program/ subprogram architecture are distributed across multiple computers on a network.

FIGURE 13.3
Main program/ subprogram architecture



5. **Layered Architecture:** The basic structure of a layered architecture . A number of different layers are defi ned, each accomplishing operations that progressively become closer to the machine instruction set. At the outer layer, components service user interface operations. At the inner layer, components perform operating system interfacing. Intermediate layers provide utility services and application software functions.